ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO GIỮA KỲ

MÔN: LẬP TRÌNH ROBOT VỚI ROS

Robot tracker với 2 link xoay sử dụng Encoder Lidar Camera

Sinh viên thực hiện: **Phạm Thành Đông - 23020735**

Giảng viên phụ trách: **TS. Lê Xuân Lực**
KS. Nguyễn Thiên Hảo

Mục lục

1 Thiết kế robot **2**
1.1 Dạng robot và thiết kế trên SolidWorks 2
1.2 Động học 3
1.2.1 Động học hệ thống di chuyển (Mobile Base Kinematics) 3
1.2.2 Động học tay máy (Manipulator Kinematics) 4

2 Đặt trục tọa độ cho file urdf **5**

3 Xây dựng mô hình mô phỏng (URDF & Gazebo) **6**
3.1 Mô tả cấu trúc file URDF 6
3.1.1 Cụm thân gầm và hệ thống di chuyển: 7
3.1.2 Hệ thống cơ cấu chấp hành: 7
3.1.3 Hệ thống cảm biến: 7
3.2 Mô hình Robot trong trình mô phỏng Gazebo và Rviz: 8
3.2.1 Mô hình Robot trong môi trường Gazebo: 8
3.2.2 Mô hình Robot trong môi trường Rviz: 9

4 Hệ thống điều khiển Robot: **10**
4.1 Cơ chế điều khiển trên Gazebo: 10
4.1.1 Điều khiển động cơ: 10
4.1.2 Điều khiển tay máy: 10
4.2 Kết quả điều khiển động cơ và tay máy: 11

5 Các loại cảm biến trong mô hình Robot: **12**
5.1 Cảm biến Encoder: 12
5.1.1 Cơ chế hoạt động: 12
5.1.2 Kết quả hoạt động: 13
5.2 Cảm biến LiDAR: 13
5.2.1 Cơ chế hoạt động: 13
5.2.2 Kết quả hoạt động: 14
5.3 Camera: 14
5.3.1 Cơ chế hoạt động: 14
5.3.2 Kết quả hoạt động: 15

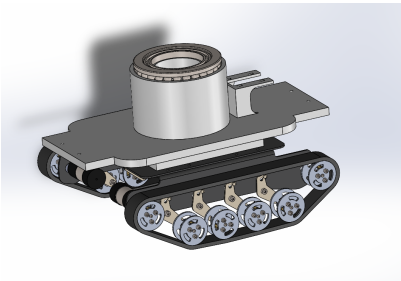
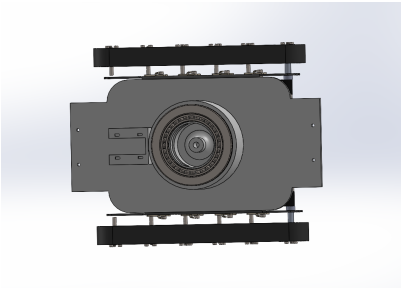
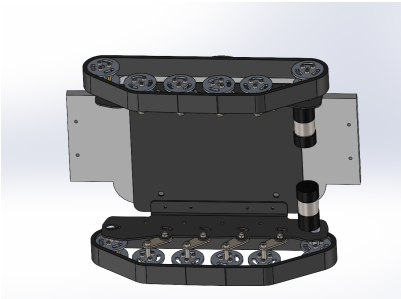
6 Kết luận **15**

1. Thiết kế robot

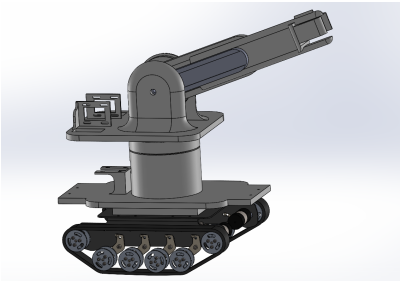
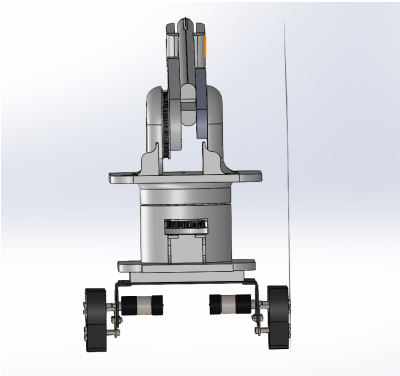
1.1. Dạng robot và thiết kế trên SolidWorks

Robot được thiết kế thuộc nhóm Mobile Manipulator (Robot tự hành kết hợp tay máy), bao gồm 2 phần chính:

1. **Hệ thống di chuyển (Mobile):** Robot sử dụng cấu hình di chuyển dạng bánh xích với một bánh mỗi bên được gắn động cơ cùng với encoder, các bánh còn lại là các bánh trợ lực. Phần khung sườn đỡ phần manipulator cùng với 2 lidar.

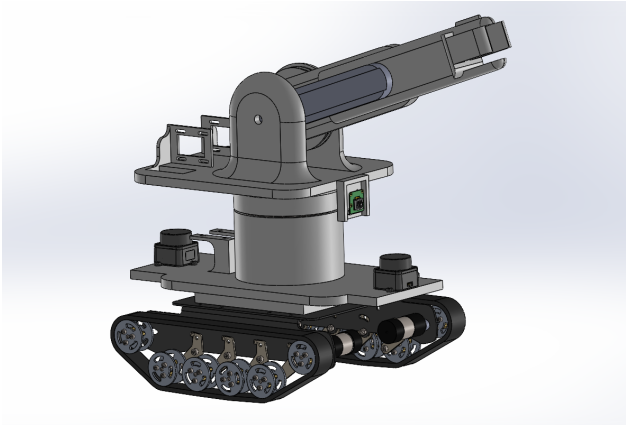




2. **Phần Manipulator:** Cánh tay robot có 2 bậc tự do loại xoay được gắn với khung của bánh xích. Khâu 1 xoay quanh trục thẳng đứng và khâu 2 là khớp gập.

3. **Tổng quan về thiết kế:**

   - Kích thước: Robot có tổng chiều dài 491,6 mm, chiều rộng 349,26 mm, chiều cao 272,1 mm.
   - Phần cảm biến: Bộ mã hóa cảm biến được gắn cùng với động cơ, có 2 lidar được sử dụng gắn trên bệ bệ của robot, camera được gắn với bệ xoay (khâu 1) giúp công việc quan sát dễ dàng hơn.



Hình 1: Robot đủ thành phần

1.2. Động học

1.2.1. Động học hệ thống di chuyển (Mobile Base Kinematics)

Mặc dù robot sử dụng cơ cấu cơ khí dạng bánh xích (Tracked Vehicle), nhưng về mặt động học mô phỏng, hệ thống được mô hình hóa theo nguyên lý Lái trượt (Skid-Steering) tương đương với Dẫn động vi sai (Differential Drive).

- **Định nghĩa các tham số và biến trạng thái:**
  - R: Bán kính bánh xe phát động.
  - L: Khoảng cách giữa hai vệt xích / tâm hai bánh lái.
  - ω_R, ω_L: Vận tốc góc của động cơ bánh phải và bánh trái (rad/s).
  - v_R, v_L: Vận tốc dài tại điểm tiếp xúc của bánh phải và bánh trái với mặt đường, trong đó $v_R = R \cdot \omega_R$ và $v_L = R \cdot \omega_L$.
  - v, ω: Vận tốc tịnh tiến dài và vận tốc góc của khối tâm robot.
  - x, y, θ: Tọa độ không gian và góc hướng của khối tâm robot trong hệ tọa độ toàn cục.
- **Phương trình Động học nội tại (Robot-centric Kinematics):**

$$v = \frac{v_R + v_L}{2} = \frac{R(\omega_R + \omega_L)}{2}$$

$$\omega = \frac{v_R - v_L}{L} = \frac{R(\omega_R - \omega_L)}{L}$$

- **Động học thuận (Forward Kinematics):**

$$\begin{cases} \dot{x} = v\cos\theta \\ \dot{y} = v\sin\theta \\ \dot{\theta} = \omega \end{cases}$$

Viết dưới dạng ma trận Jacobian thay thế v và ω từ phương trình nội tại:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{R}{2}\cos\theta & \frac{R}{2}\cos\theta \\ \frac{R}{2}\sin\theta & \frac{R}{2}\sin\theta \\ \frac{R}{L} & -\frac{R}{L} \end{bmatrix} \begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix}$$

- **Động học nghịch (Inverse Kinematics):**

  Từ hệ phương trình nội tại, ta giải hệ phương trình hai ẩn v_R và v_L:

$$v_R = v + \frac{\omega \cdot L}{2}$$

$$v_L = v - \frac{\omega \cdot L}{2}$$

  Chia cho bán kính R, ta thu được công thức vận tốc góc cấp cho động cơ:

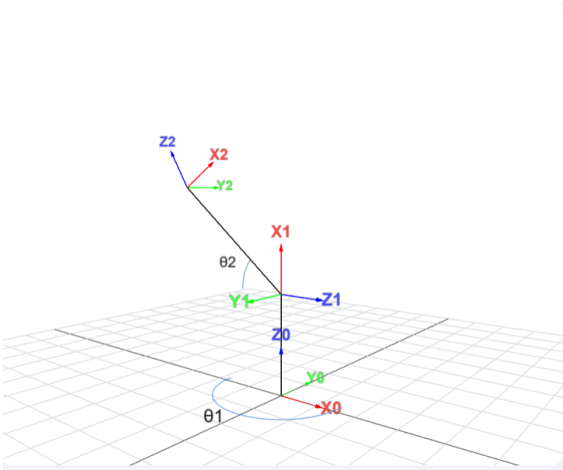
$$\omega_R = \frac{1}{R}\left(v + \frac{\omega \cdot L}{2}\right)$$

$$\omega_L = \frac{1}{R}\left(v - \frac{\omega \cdot L}{2}\right)$$

1.2.2. Động học tay máy (Manipulator Kinematics)

Tay máy gồm 2 bậc tự do (2-DOF) gồm 1 đế xoay và 1 khớp gập. Ta xây dựng hệ trục tọa độ theo quy tắc Denavit-Hartenberg (D-H).

- **Hệ trục** 0 **(**X_0, Y_0, Z_0**):** Gắn tại đế (Base), với trục Z_0 hướng thẳng đứng lên trên dọc theo trục xoay của khớp 1.
- **Hệ trục** 1 **(**X_1, Y_1, Z_1**):** Gắn tại khớp gập, với trục Z_1 nằm ngang, vuông góc với Z_0 và là trục quay của khớp 2.
- **Hệ trục** 2 **(**X_2, Y_2, Z_2**):** Gắn tại điểm tác động cuối (End-Effector) của cánh tay.



Hình 2: Hệ trục tọa độ D-H

Bảng tham số D-H:

- **Phương trình tọa độ điểm cuối (Forward Kinematics Equations)**

  Từ bảng D-H trên, ta lập các ma trận chuyển đổi thuần nhất A_i từ hệ tọa độ $i-1$ sang hệ i:

Bảng 1: Bảng thông số Denavit-Hartenberg (DH)

Khâu (i)	**Góc quay (θ_i)**	**Độ lệch (d_i)**	**Chiều dài (a_i)**	**Góc xoắn (α_i)**
1	θ_1	L_1	0	$90°$
2	θ_2	0	L_2	$0°$

- **Ma trận chuyển đổi Khâu 1 (${}^0A_1$):**

$$ {}^0A_1 = \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & 0 \\ \sin\theta_1 & 0 & -\cos\theta_1 & 0 \\ 0 & 1 & 0 & L_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

- **Ma trận chuyển đổi Khâu 2 (${}^1A_2$):**

$$ {}^1A_2 = \begin{bmatrix} \cos\theta_2 & -\sin\theta_2 & 0 & L_2\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & 0 & L_2\sin\theta_2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

- **Ma trận chuyển đổi toàn cục (${}^0T_2$):**

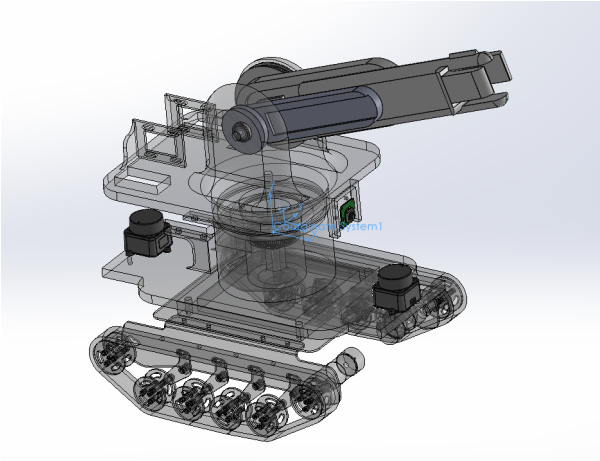
$$ {}^0T_2 = {}^0A_1{}^1A_2 = \begin{bmatrix} \cos\theta_1\cos\theta_2 & -\cos\theta_1\sin\theta_2 & \sin\theta_1 & L_2\cos\theta_1\cos\theta_2 \\ \sin\theta_1\cos\theta_2 & -\sin\theta_1\sin\theta_2 & -\cos\theta_1 & L_2\sin\theta_1\cos\theta_2 \\ \sin\theta_2 & \cos\theta_2 & 0 & L_1 + L_2\sin\theta_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

- **Ta thu được tọa độ cuối:**

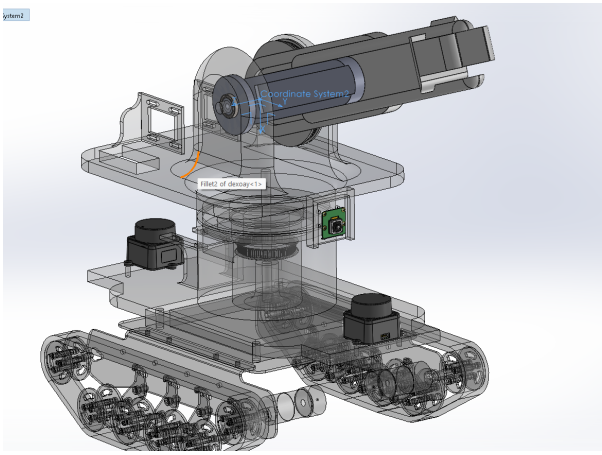
$$ \begin{bmatrix} X_e \\ Y_e \\ Z_e \end{bmatrix} = \begin{bmatrix} L_2\cos\theta_1\cos\theta_2 \\ L_2\sin\theta_1\cos\theta_2 \\ L_1 + L_2\sin\theta_2 \end{bmatrix} $$

2. Đặt trục tọa độ cho file urdf

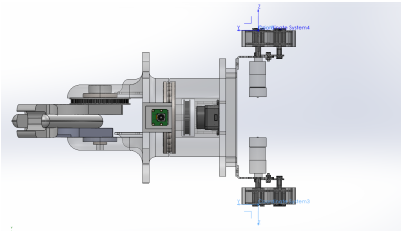
Các hệ trục tọa độ được đặt vào các vật thể chuyển động (Động cơ, các khâu). Phần cảm biến cũng được đặt hệ trục để dễ dàng điều chỉnh cảm biến trong gazebo và rviz. Trong hệ trục, trục Z được hướng theo chiều chuyển động của vật.



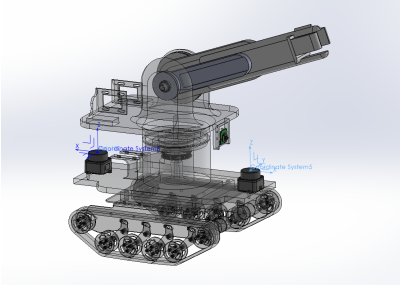
Hình 3: Trục tọa độ cho khâu 1



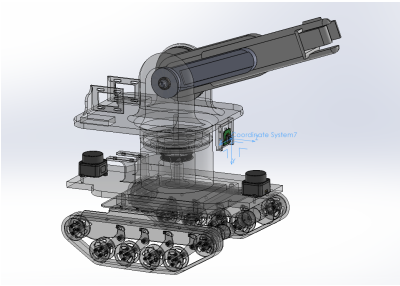
Hình 4: Trục tọa độ cho khâu 2



Hình 5: Trục tọa độ động cơ và encoder



Hình 6: Trục tọa độ cho lidar



Hình 7: Trục tọa độ cho camera

3. Xây dựng mô hình mô phỏng (URDF & Gazebo)

3.1. Mô tả cấu trúc file URDF

Mô hình robot được xây dựng trên định dạng URDF nhằm phục vụ công tác mô phỏng động lực học bằng các phần mềm mô phỏng như Gazebo. Cấu trúc này được phân định rõ ràng thành hệ thống khung gầm di động kết hợp với một hệ cơ cấu chấp hành và nhận thức môi trường đặt ở phía trên.
Các bộ phận và khớp nối được liên kết với nhau theo một sơ đồ phân cấp dạng cây:

```
[base_link] (Khung gầm trung tâm)
├── [banhtrai_joint] ⟶ [banhtrai_link] (Hệ chuyển động)
├── [banhphai_joint] ⟶ [banhphai_link] (Hệ chuyển động)
├── [lidar1_joint] ⟶ [lidar1] (Hệ thống nhận thức 1)
├── [lidar2_joint] ⟶ [lidar2] (Hệ thống nhận thức 2)
└── [dexoay_joint] ⟶ [dexoay] (Đế mâm xoay hướng)
    ├── [khau1_joint] ⟶ [khau1] (Khâu nâng hạ)
    └── [camera_joint] ⟶ [camera] (Hệ thống thị giác)
```

3.1.1. Cụm thân gầm và hệ thống di chuyển:

- **Base Link (Khung gầm chính):** Đóng vai trò là gốc tọa độ của toàn bộ mô hình. Vật thể có khối lượng tính toán là 2.31 kg. Trong đó, khối tâm (origin xyz) của `base_link` được đặt lệch khá xa so với mốc 0 ban đầu (Y = -1.2988), phản ánh hệ quy chiếu toàn cục khi thiết kế CAD chưa được đặt tại trung tâm của robot.

- **Hệ thống bánh lái (`banhtrai_link` và `banhphai_link`):** Hai bánh xe có khối lượng tương đồng (khoảng 0.045 kg/bánh) và được liên kết với khung gầm thông qua các khớp loại continuous (khớp quay liên tục). Trong khai báo vị trí, cả hai bánh đều được áp dụng các phép xoay góc 180°(rpy="0 0 -3.1416" và rpy="-3.1416 0 0") nhằm đảm bảo mặt vành của bánh xe cùng hướng ra phía ngoài khung gầm. Khoảng cách chênh lệch tọa độ Z giữa hai khớp này cho thấy chiều rộng cơ sở của xe xấp xỉ 0.266 mét.

3.1.2. Hệ thống cơ cấu chấp hành:

- **Đế xoay (`dexoay`):** Là bệ đỡ trung tâm cho phần thân trên, khối lượng 1.63 kg, liên kết với `base_link` qua khớp revolute (khớp quay có giới hạn). Khớp này hoạt động quanh trục Z (xyz="0 0 1"), cho phép toàn bộ phần trên có thể thực hiện các chuyển động xoay ngang (Yaw motion).

- **Khâu cánh tay (`khau1`):** Khâu chuyển động số 1 nặng khoảng 0.9 kg, được nối tiếp trực tiếp lên bộ phận `dexoay` thông qua khớp `khau1_joint`. Khớp này cho phép khâu thực hiện chuyển động gập/nâng với biên độ thiết kế là 180° (từ 90° đến +90°, tương ứng -1.57 đến 1.57 radian).

3.1.3. Hệ thống cảm biến:

- **Cụm LiDAR:** Robot được trang bị hai mô-đun Lidar (`lidar1` và `lidar2`) gắn đối xứng (hoặc trước/sau) trên khung xe cơ sở thông qua khớp fixed (cố định hoàn toàn). Các mô-đun này đã được định tuyến xoay một góc 90° (1.5708 rad) để tối ưu hóa góc quét không gian 3D xung quanh robot. Khối lượng mỗi cảm biến khá nhẹ, xấp xỉ 12 gram.

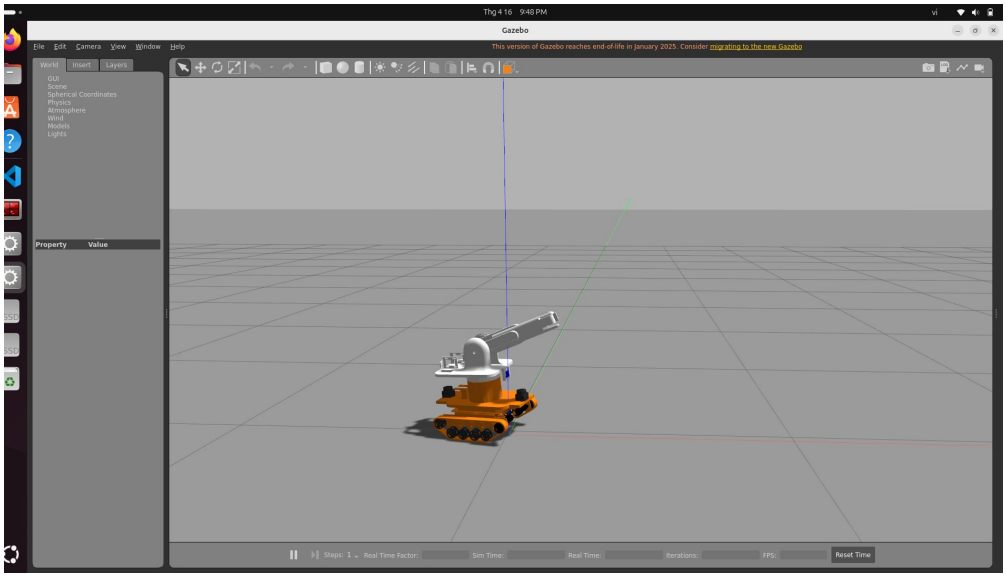
- **Camera:** Thiết bị thị giác được gắn cố định (fixed) lên bộ phận `dexoay` với độ nhô về phía trước khoảng 0.1 mét. Do Camera nhận `dexoay` làm Parent thay vì `khau1`, thiết bị này sẽ chỉ quét theo phương ngang khi đế xoay chuyển động, và hoàn toàn không bị ảnh hưởng bởi góc ngước/cúi của cánh tay `khau1`.

- **Encoder:** Trên hệ thống vật lý thực tế, robot được trang bị cảm biến Encoder gắn tại các trục động cơ (bánh trái và bánh phải) nhằm đo lường vận tốc và góc quay của bánh xe, phục vụ cho bài toán ước lượng vị trí (Odometry). Tuy nhiên, xét về mặt cấu trúc không gian trong tệp định dạng URDF, Encoder không được biểu diễn dưới dạng một khối hình học (Link) hay một khớp (Joint) độc lập. Thay vì là một vật thể vật lý, chức năng của Encoder được trừu tượng hóa và nội suy trực tiếp từ trạng thái của hai khớp truyền động chính là `banhtrai_joint` và `banhphai_joint`.

Trong quá trình mô phỏng, tín hiệu Encoder sẽ được hệ thống phần mềm (Gazebo Plugins) giả lập bằng cách liên tục trích xuất dữ liệu vòng quay tại hai khớp này (thông qua JointState). Dữ liệu này sau đó được đẩy vào bộ giải thuật động học (Differential Drive Controller) để tính toán vận tốc tuyến tính, vận tốc góc và lập bản đồ quỹ đạo di chuyển của toàn bộ khung gầm (`base_link`) theo thời gian thực.

3.2. Mô hình Robot trong trình mô phỏng Gazebo và Rviz:

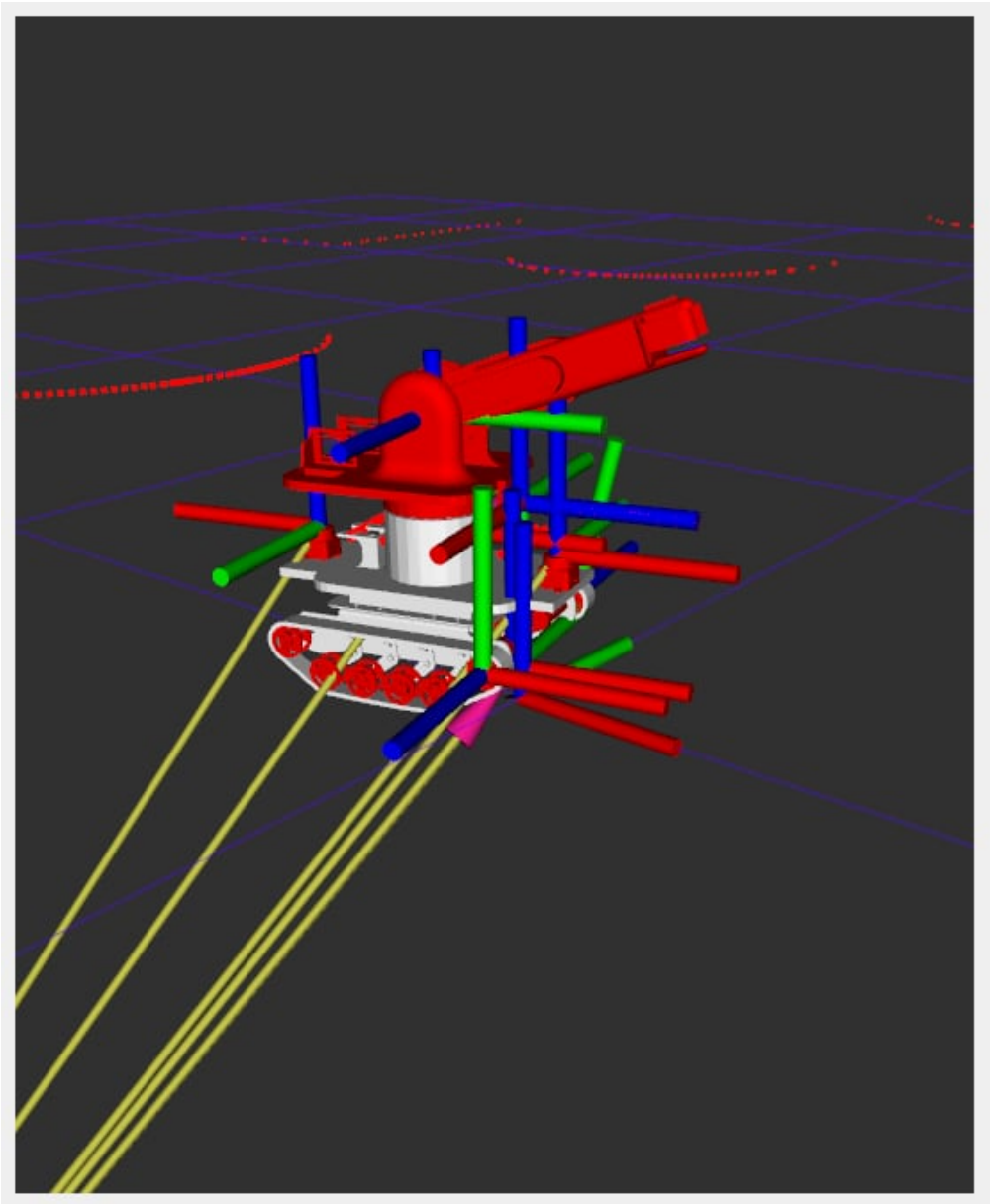
3.2.1. Mô hình Robot trong môi trường Gazebo:



Nhận xét:

- Gazebo đã render thành công các file .STL của robot. Màu sắc nguyên bản từ CAD (Cam ở khung gầm, Trắng ở phần thân trên và tay máy) được giữ nguyên, cho thấy thẻ `<material>` trong URDF hoạt động tốt.
- Khác với tên gọi `banhtrai_link` hay `banhphai_link` trong code URDF, hình ảnh cho thấy robot sử dụng hệ thống bánh xích (track) cùng các bánh tì nhỏ. Điều này cho thấy trong URDF, cụm bánh xích này đang được mô hình hóa đơn giản hóa dưới dạng cơ cấu Differential Drive (2 bánh hai bên) để giảm tải tính toán vật lý, nhưng phần hiển thị (Visual) vẫn giữ nguyên hình dáng bánh xích phức tạp.
- Việc robot đứng vững trên mặt phẳng tọa độ (Grid) và có đổ bóng (Shadow) chứng minh rằng các thông số về khối lượng (Mass), quán tính (Inertia) và đặc biệt là thẻ va chạm `<collision>` đã được thiết lập, đồng thời hệ thống không bị rơi xuyên qua mặt đất hay sụp đổ cấu trúc khớp.

3.2.2. Mô hình Robot trong môi trường Rviz:



Nhận xét:

- **Hệ thống trục tọa độ:** Các cụm trục tọa độ 3 màu (Đỏ - X, Xanh lá - Y, Xanh dương - Z) xuất hiện dày đặc tại vị trí khung gầm, hệ thống bánh xích, mâm xoay và camera. Điều này chứng minh cây động học (Kinematic Tree) khai báo trong tệp URDF đã được liên kết đúng logic không gian.
- **Trực quan hóa dữ liệu cảm biến:** Các vệt chấm màu đỏ trên mặt phẳng là dữ liệu không gian (LaserScan/Point Cloud) được thu thập từ mô-đun Lidar và khối chóp nón màu vàng chiếu ra từ đầu cánh tay robot chính là biểu diễn trường nhìn (Field of View) của Camera.

4. Hệ thống điều khiển Robot:

4.1. Cơ chế điều khiển trên Gazebo:

4.1.1. Điều khiển động cơ:

Để điều hướng cơ cấu truyền động vi sai của robot, hệ thống sử dụng gói công cụ tiêu chuẩn `teleop_twist_keyboard` của ROS 2. Gói này có nhiệm vụ thu thập phím bấm của người dùng và chuyển đổi thành thông điệp dạng `geometry_msgs/Twist` chứa các giá trị vận tốc tuyến tính (linear) và vận tốc góc (angular), sau đó phát (publish) liên tục lên topic `/cmd_vel` để trình giả lập xử lý.

Để vận hành điều khiển node này, ta kích hoạt môi trường và chạy node điều khiển tại Terminal:

```
source /opt/ros/humble/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

4.1.2. Điều khiển tay máy:

Khác với khung gầm sử dụng gói có sẵn, hệ thống tay máy 2 bậc tự do (2-DOF) được điều khiển qua một bộ giải thuật tự phát triển, áp dụng kiến trúc Service Client-Server trong ROS 2. Phân hệ này bao gồm hai nút (node) chính hoạt động song song:

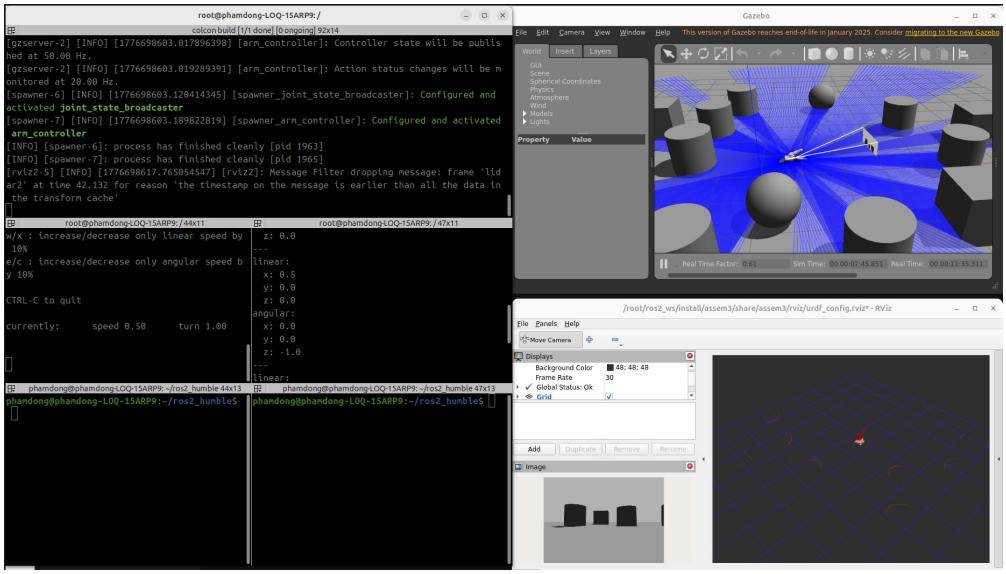
- **Nút Giao diện Người dùng (`arm_teleop_key`):** Có nhiệm vụ lắng nghe tín hiệu từ bàn phím ở chế độ thô (raw mode - không cần nhấn Enter) thông qua các thư viện hệ thống (termios, tty). Mỗi lần nhấn phím tương ứng với một bước nhảy góc là 5°. Để đảm bảo an toàn cơ khí (tránh va chạm tĩnh), thuật toán đã tích hợp bộ khóa giới hạn (Clamping), ép giá trị góc của trục Yaw (đế xoay) trong biên độ [−180°, 180°] và trục Pitch (khâu vươn) trong biên độ [−90°, 90°] đúng với ràng buộc URDF.

- **Nút Xử lý Động học (`arm_control_node`):** Có nhiệm vụ tiếp nhận góc quay mục tiêu từ Client. Tại đây, hệ thống tận dụng linh hoạt cấu trúc thông điệp chuẩn `AddTwoInts` để truyền dữ liệu (tham số a cho khớp đế xoay, b cho khớp gập). Thuật toán tự động chuyển đổi đơn vị từ Độ (Degree) sang Radian. Sau đó, nó thiết lập một thông điệp dạng `JointTrajectory` để điều khiển đồng thời hai khớp (`dexoay_joint`, `khau1_joint`). Nhằm tránh hiện tượng giật cục gây hỏng phần cứng ảo, tham số `time_from_start` được cấu hình là 2 giây. Dữ liệu này được publish lên topic `/arm_controller/joint_trajectory`, ra lệnh cho bộ điều khiển mô phỏng thực hiện phép nội suy quỹ đạo trơn tru trong đúng 2 giây để đạt tới góc mục tiêu.

Thao tác vận hành tay máy:

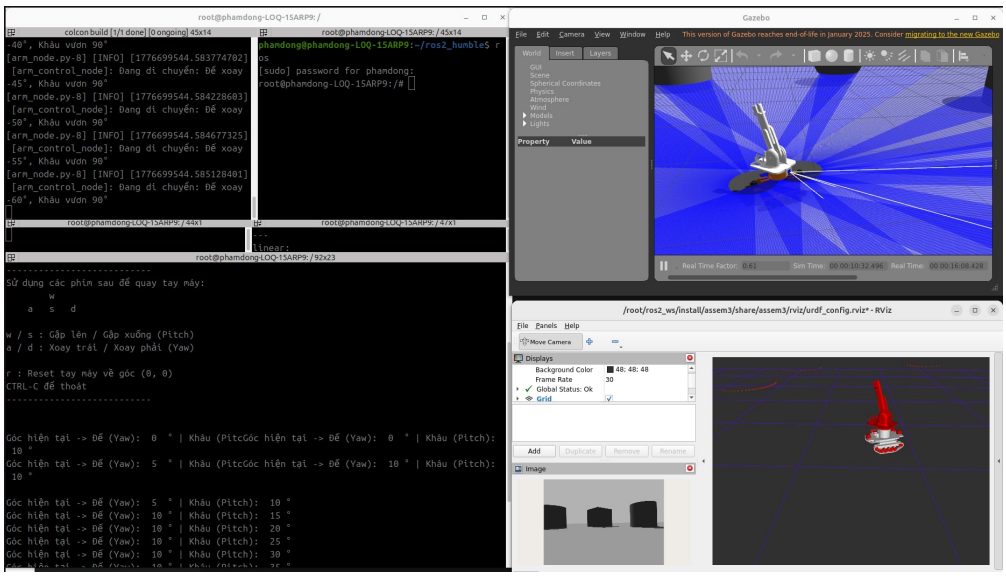
```
source ~/ros2_ws/install/setup.bash
ros2 run assem3 arm_teleop_key.py
```

4.2. Kết quả điều khiển động cơ và tay máy:



Nhận xét kết quả điều khiển động cơ:

- Giao diện bàn phím hoạt động ổn định, truyền thông điệp `cmd_vel` chính xác với vận tốc tiến (linear.x: 0.5) và chuyển hướng rẽ phải (angular.z: -1.0).
- Robot di chuyển mượt mà trong sa bàn chướng ngại vật. Các tia quét Lidar (xanh dương) và trường nhìn Camera (nón trắng) tương tác hoàn hảo với các khối vật thể xung quanh.
- Hệ thống lập bản đồ chính xác biên dạng vật cản qua đám mây điểm Lidar (vệt chấm đỏ). Đồng thời, luồng video từ Camera (cửa sổ Image góc dưới trái) truyền về rõ nét góc nhìn thứ nhất của robot.



Nhận xét kết quả điều khiển tay máy:

- Giao diện điều khiển từ bàn phím (`arm_teleop_key`) phản hồi tức thì, cập nhật liên tục giá trị góc Yaw và Pitch trên terminal, giúp người dùng kiểm soát chính xác tư thế tay máy.
- Node trung gian (`arm_control_node`) đã giải mã lệnh thành công, chuyển đổi dữ liệu từ dạng số nguyên (độ) sang thông điệp quỹ đạo (Radian) để truyền tới bộ điều khiển khớp.
- Trong Gazebo, tay máy thực hiện các cử động gập và xoay trơn tru, đúng với giá trị góc lệnh. Trong RViz, trạng thái hình học của robot được cập nhật đồng bộ, cho phép theo dõi trực quan vị trí cánh tay so với khung gầm.

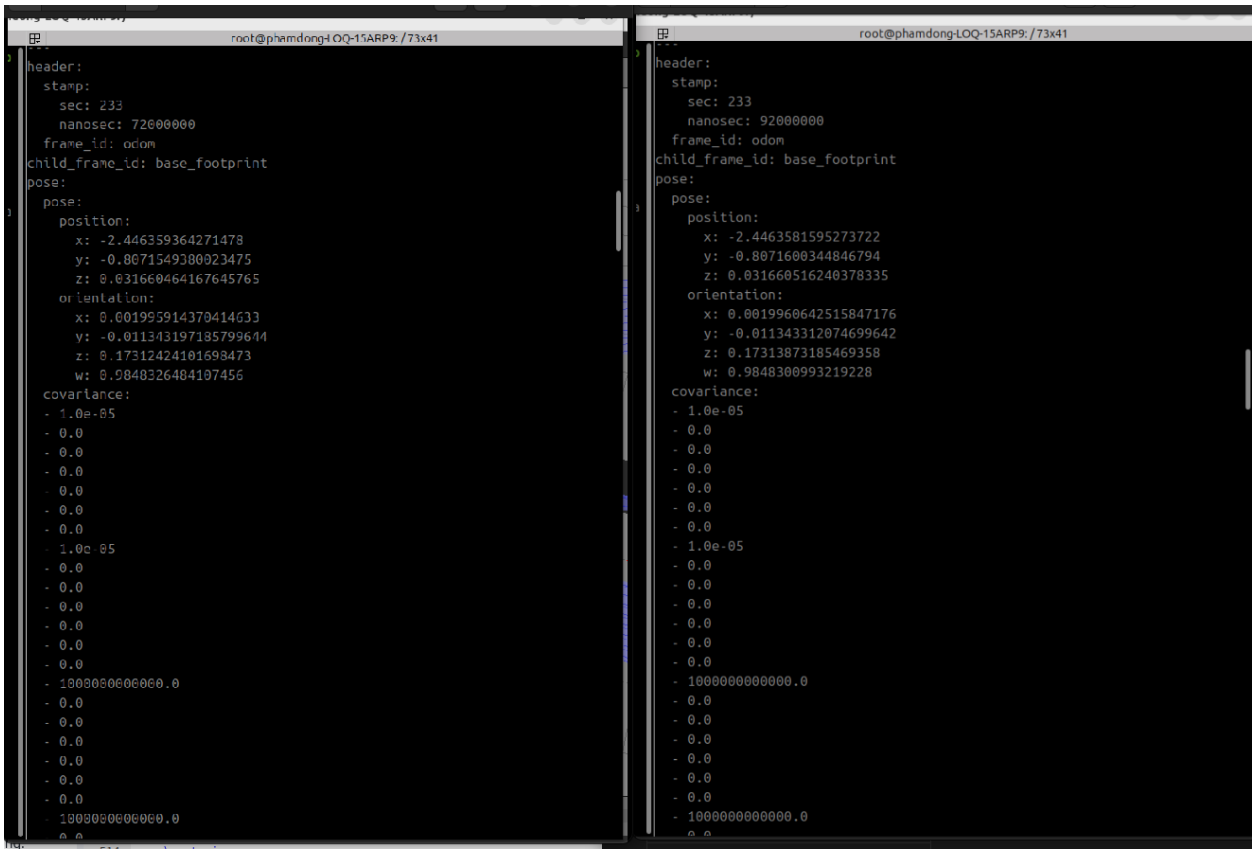
5. Các loại cảm biến trong mô hình Robot:

5.1. Cảm biến Encoder:

5.1.1. Cơ chế hoạt động:

- **Giả lập vật lý:** Trong môi trường Gazebo, mỗi khớp truyền động (`banhtrai_joint`, `banhphai_joint`) được gán các thuộc tính vật lý và giới hạn chuyển động. Bộ giải vật lý (Physics Engine - ODE) sẽ mô phỏng việc quay thực tế của các bánh xe dựa trên lệnh từ topic `/cmd_vel`.
- **Trích xuất dữ liệu:** Node `joint_state_broadcaster` liên tục truy vấn trạng thái thời gian thực của các khớp này, trích xuất giá trị góc quay (position) và tốc độ góc (velocity) của trục động cơ tại mỗi chu kỳ lặp (thường là 50Hz - 100Hz).
- **Phản hồi hệ thống:** Dữ liệu từ encoder ảo được đóng gói vào thông điệp `sensor_msgs/JointState` và phát hành lên hệ thống. Các bộ điều khiển điều hướng sẽ đăng ký (subscribe) vào dữ liệu này để thực hiện phép toán Odometry, từ đó ước lượng quãng đường robot đã di chuyển.

5.1.2. Kết quả hoạt động:



- Với tần suất hoạt động 50Hz (như ghi nhận trong log hệ thống), hệ thống cung cấp dữ liệu phản hồi vị trí với sai số cực thấp, đủ để duy trì quỹ đạo di chuyển ổn định mà không bị trượt (slip) trong môi trường mô phỏng.
- Dữ liệu từ Encoder ảo đảm bảo tính đồng bộ với hệ trục tọa độ (TF Tree) của robot, giúp các cảm biến Lidar và Camera có được dữ liệu tĩnh tương đối so với thế giới xung quanh, là điều kiện tiên quyết cho các bài toán bản đồ (SLAM).

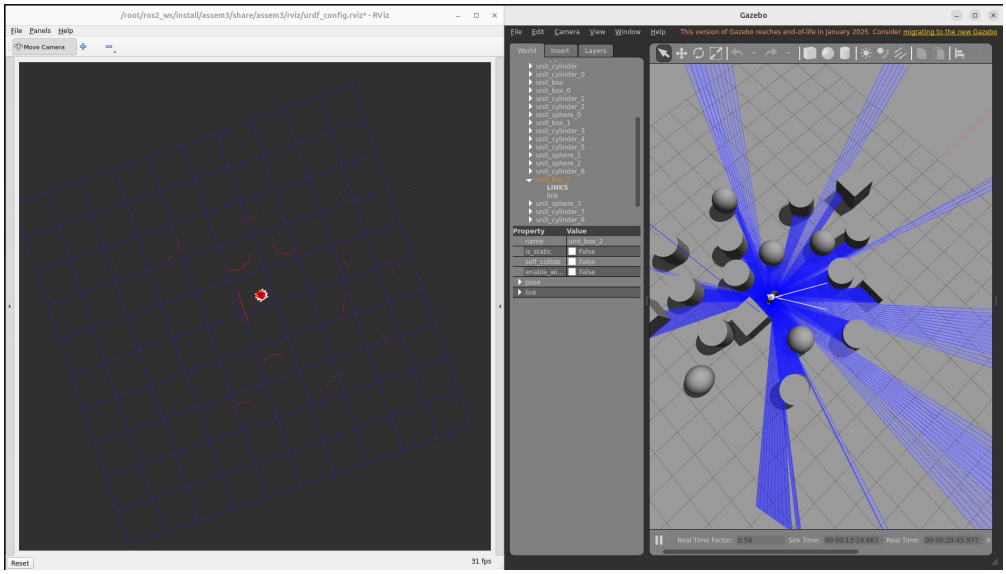
5.2. Cảm biến LiDAR:

5.2.1. Cơ chế hoạt động:

Cảm biến LiDAR hoạt động dựa trên nguyên lý quét tia laser theo thời gian thực (Time-of-Flight):

- Hệ thống LiDAR ảo trong Gazebo phát ra các dải tia laser (hiển thị bằng các tia xanh dương tỏa ra 360 độ quanh robot). Các tia này khi gặp chướng ngại vật sẽ bị phản xạ ngược lại cảm biến.
- Dựa trên độ trễ thời gian của tia phản xạ, cảm biến tính toán chính xác tọa độ (x,y,z) của điểm va chạm so với tâm robot (`base_link`).
- Dữ liệu này được chuyển đổi thành thông điệp `sensor_msgs/LaserScan` hoặc PointCloud2 và truyền tải vào hệ thống thông qua các topic ROS.

5.2.2. Kết quả hoạt động:



Nhận xét kết quả hoạt động:

- Trong cửa sổ Gazebo, các tia LiDAR quét xuyên suốt không gian sa bàn, nhận diện chính xác các khối trụ và bức tường. Trong cửa sổ RViz, các chướng ngại vật này được tái hiện lại dưới dạng các đường cong điểm đỏ (đám mây điểm) bám sát theo biên dạng của vật thể thực.

- Hình ảnh cho thấy khi tay máy di chuyển hoặc khi robot thay đổi vị trí, sơ đồ điểm (Point Cloud) trong RViz được cập nhật liên tục. Điều này khẳng định hệ thống có độ trễ cực thấp, đáp ứng tiêu chuẩn thời gian thực cho các ứng dụng điều hướng.

- Sự đồng nhất giữa dữ liệu LiDAR (đám mây điểm) và dữ liệu hình ảnh từ Camera (khối hộp màu đen trong cửa sổ Image) cho thấy robot đã hoàn thiện khả năng "nhìn" đa tầng: LiDAR giúp đo khoảng cách và tránh va chạm, trong khi Camera cung cấp thông tin trực quan để nhận diện vật thể.

5.3. Camera:

5.3.1. Cơ chế hoạt động:

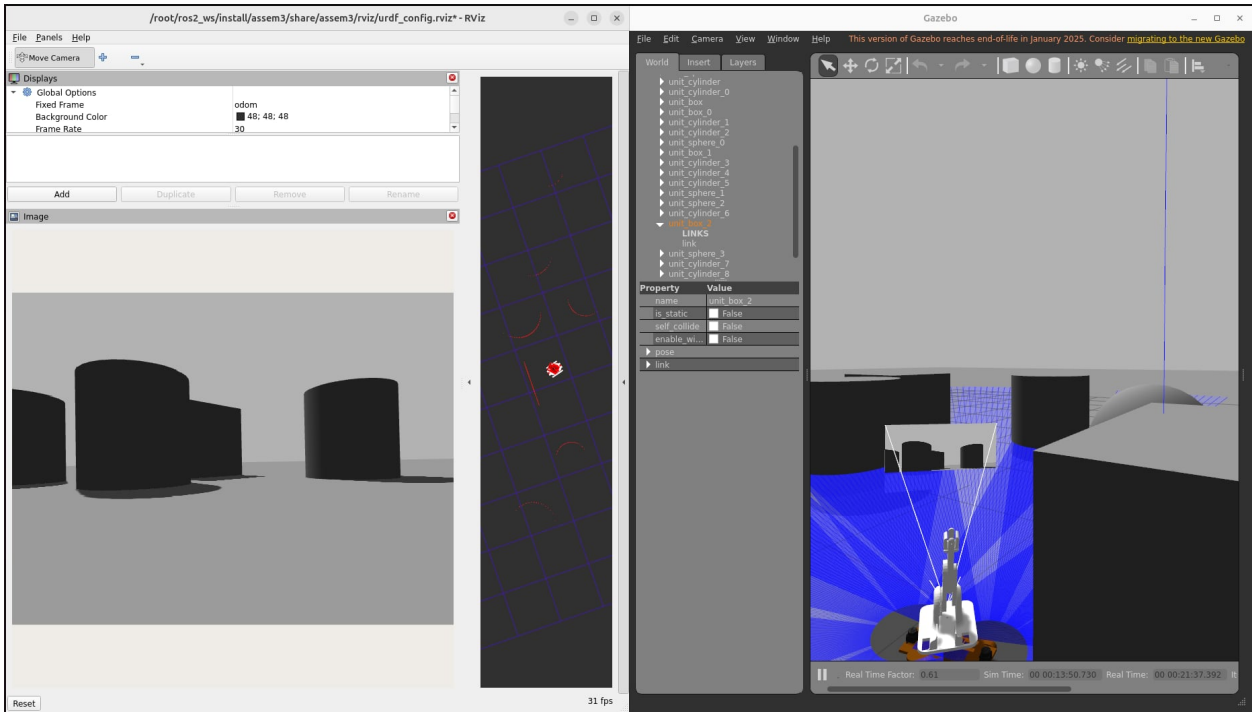
- Camera được mô hình hóa trong Gazebo dưới dạng một cảm biến hình ảnh số (virtual camera sensor) với các thông số tiêu cự, độ phân giải và trường nhìn (FOV) được gán trong URDF.

- Camera thực hiện chụp liên tục khung hình từ không gian 3D của Gazebo. Luồng dữ liệu thô (raw image) này sau đó được mã hóa và truyền tải thông qua giao thức ROS 2 dưới dạng topic `sensor_msgs/Image`.

- Nhờ liên kết với `dexoay_joint` và `khau1_joint`, Camera luôn cập nhật được vị trí và hướng nhìn trong không gian 3D. Mỗi thay đổi về góc quay của đế xoay

hay khâu gập tay máy đều được hệ thống TF (Transform) tự động tính toán để phản hồi chính xác hướng quan sát của camera.

5.3.2. Kết quả hoạt động:



Nhận xét kết quả hoạt động của camera:

- Cửa sổ Image trong RViz hiển thị hình ảnh đen trắng (grayscale) của các vật cản (khối trụ, khối hộp) với độ tương phản tốt và góc nhìn đồng nhất với vị trí thực tế của robot trong sa bàn Gazebo.
- Hình ảnh cho thấy Camera nắm bắt thành công các vật cản ngay trước mặt, dù robot đang ở vị trí xoay hay gập tay máy. Độ phân giải hiển thị rõ nét biên dạng của vật thể, đủ điều kiện để triển khai các thuật toán xử lý ảnh tiếp theo như nhận diện cạnh (Canny edge detection), nhận diện hình dạng (Shape recognition) hoặc phân loại đối tượng.

6. Kết luận

Sau quá trình thiết kế và thực hiện mô phỏng hệ thống robot di động có tay máy, dự án đã hoàn thành các mục tiêu đề ra và đạt được những kết quả cụ thể. Về thiết kế và cấu trúc, ta đã xây dựng thành công mô hình robot 3D từ giai đoạn thiết kế trên SolidWorks đến việc chuẩn hóa tệp URDF để tương thích hoàn toàn với hệ sinh thái ROS 2. Cấu trúc động học được thiết lập chặt chẽ với các khớp nối linh hoạt, đảm bảo tính chính xác trong việc mô phỏng các chuyển động phức tạp của cả khung gầm di động và hệ thống tay máy 2-DOF. Hệ thống đã chứng minh được khả năng đáp ứng thời gian thực thông qua các thử nghiệm điều khiển trên môi trường mô

phỏng Gazebo và RViz. Việc triển khai các nút (nodes) điều khiển di chuyển và dịch vụ (services) điều khiển tay máy cho thấy tính ổn định cao. Người vận hành có thể điều khiển robot một cách trực quan, mượt mà thông qua giao diện dòng lệnh và phản hồi dữ liệu từ hệ thống. Hệ thống đồng thời cũng tích hợp thêm các loại cảm biến, ung cấp dữ liệu môi trường chính xác phục vụ cho các bài toán nhận diện và định vị. Cơ chế giả lập Encoder thông qua joint_state_broadcaster cũng cung cấp nền tảng vững chắc cho các thuật toán đo lường vị trí (Odometry) trong tương lai.

Mã nguồn dự án:
Toàn bộ mã nguồn mô phỏng (ROS 2), các cấu hình hệ thống (URDF, Launch files):
https://github.com/DOGDOT/giuakyros/tree/main